Git Ultimate Cheat Sheet

Syntax, Options, and Practical Examples for Every Workflow

⚙️ 1. Setup & Configuration

Configure Git Settings - Set up your user details and preferences.

GENERAL SYNTAX

```
git config [--global | --local] <setting.name> "<value>"
```

EXAMPLE

```
git config --global user.name "Ada Lovelace"
# Sets the global name for all commits on this machine
```

🚀 2. Starting a Project

Initialize Repository - Create a new local repository.

GENERAL SYNTAX

```
git init [<directory_name>]
```

EXAMPLE

```
git init my-awesome-app
# Creates a folder named 'my-awesome-app' and initializes Git inside it
```

Clone Repository - Download an existing project.

GENERAL SYNTAX

```
git clone <repository_url> [<local_folder_name>]
```

EXAMPLE

```
git clone https://github.com/user/repo.git my-repo
# Clones the repo into a specific local folder named 'my-repo'
```

📝 3. Making Changes

Stage Changes - Prepare files to be committed.

GENERAL SYNTAX

```
git add <path_spec>
```

EXAMPLE

```
git add index.html src/*.js
# Stages index.html and all JavaScript files in the src folder
```

Commit Changes - Save your staged changes to history.

GENERAL SYNTAX

```
git commit [-m "<message>" | --amend]
```

EXAMPLE

```
git commit -m "Fix navigation bar styling bug"
# Commits the staged files with a clear, descriptive message
```

View Differences - See what you've changed.

GENERAL SYNTAX

```
git diff [<commit_1>] [<commit_2>] [<file_path>]
```

EXAMPLE

```
git diff main feature-branch styles.css
# Shows changes made to styles.css between the 'main' and 'feature-branch'
```

🌿 4. Branching & Merging

Manage Branches - List, create, or delete branches.

GENERAL SYNTAX

```
git branch [-d] [<branch_name>]
```

EXAMPLE

```
git branch -d old-feature
# Safely deletes the local branch named 'old-feature'
```

Switch Branches - Move between different branches.

GENERAL SYNTAX

```
git switch [-c] <branch_name>
```

EXAMPLE

```
git switch -c new-login-page
# Creates a new branch named 'new-login-page' and switches to it immediately
```

Merge Branches - Combine history from another branch into your current one.

GENERAL SYNTAX

```
git merge <source_branch>
```

EXAMPLE

```
git merge feature-update-ui
# Merges the changes from 'feature-update-ui' into your currently active branch
```

☁️ 5. Synchronizing (Remote)

Add Remote - Connect local repo to a remote server.

GENERAL SYNTAX

```
git remote add <remote_name> <repository_url>
```

EXAMPLE

```
git remote add upstream https://github.com/original/repo.git
# Adds a secondary remote named 'upstream' (useful for forks)
```

Push Changes - Upload your local commits.

GENERAL SYNTAX

```
git push [-u] <remote_name> <branch_name>
```

EXAMPLE

```
git push -u origin feature-auth
# Uploads 'feature-auth' and links it to origin so future pushes just need 'git
push'
```

Pull Changes - Download and merge changes from the remote.

GENERAL SYNTAX

```
git pull <remote_name> <branch_name>
```

EXAMPLE

```
git pull origin main
# Fetches updates from the remote 'main' branch and merges them into your local
branch
```

⏪ 6. Undoing Mistakes

Restore Files - Discard uncommitted changes.

GENERAL SYNTAX

```
git restore [--staged] <file_path>
```

EXAMPLE

```
git restore --staged config.js
# Removes config.js from the staging area (un-stages it), but keeps your edits
```

Revert Commit - Create a new commit that undoes a past commit.

GENERAL SYNTAX

```
git revert <commit_hash>
```

EXAMPLE

```
git revert 9f4a8b2
# Generates a new commit that safely reverses whatever changes 9f4a8b2
introduced
```

Reset History - Move branch pointer backwards in time.

GENERAL SYNTAX

```
git reset [--soft | --mixed | --hard] <commit_hash>
```

EXAMPLE

```
git reset --hard HEAD~3
# DANGER: Rewinds back 3 commits and DESTROYS all local changes made since then
```

🔍 7. Inspecting History

View Logs - See the commit timeline.

GENERAL SYNTAX

```
git log [--oneline | --graph | -n <number>]
```

EXAMPLE

```
git log --oneline --graph -n 10
# Shows a visual tree of the last 10 commits, with each commit on a single line
```

📦 8. Stashing

Stash Changes - Temporarily shelf uncommitted work.

GENERAL SYNTAX

```
git stash [push -m "<message>" | pop | drop | list]
```

EXAMPLE

```
git stash push -m "WIP: Header component logic"
# Shelves your current modifications under a descriptive name so you can switch
branches safely
```